Optimizing Retrieval-Augmented Generation Parameters and Comparing CPU and GPU Performance for BenepiKnow

Alexsandra Duhac
Department of Computer Studies & Engineering (CSE)
José Rizal University
Mandaluyong, Philippines
alexsandra.duhac@my.jru.edu

Anthony S. Mijares
Department of Computer Studies & Engineering (CSE)
José Rizal University
Mandaluyong, Philippines
anthony.mijares@my.jru.edu

Jude Emmanuele D. Pineda
Department of Computer Studies & Engineering (CSE)
José Rizal University
Mandaluyong, Philippines
judeemmanuele.pineda@my.jru.edu

***Abstract*—BenepiKnow is a retrieval-augmented generation (RAG) chatbot for Philippine family planning rights, benefits, administrative requirements, consent, and confidentiality. The experiment tuned the chunk size, chunk overlap, and top-k of the RAG pipeline and compared the Google Colab CPU and T4 GPU runtimes under the same workload. The knowledge base contained 528 pages from seven official documents, and the test set used five questions in English, Filipino, and Taglish. A chunk size of 800, an overlap of 150, and a top-k of 3 gave the highest Recall@3 of 0.800. The T4 GPU indexed the documents 8.76 to 10.55 times faster than the CPU runtime, while answer generation stayed between 0.69 and 1.07 times because the SEA-LION API generated the answers outside Colab. The model answered 12 of the 15 prompts correctly, and Filipino questions had the lowest retrieval quality.**

Keywords—chunk size, embedding, GPU speedup, retrieval-augmented generation, top-k

I. Introduction

Retrieval-augmented generation (RAG) improves the answers of a large language model (LLM) by first retrieving passages from a document collection and then giving the retrieved passages to the model as context [1]. BenepiKnow applies RAG to official Philippine documents on family planning so that general questions about rights, benefits, administrative requirements, consent, and confidentiality are answered from verified sources. The quality of a RAG answer depends on how the documents are split and searched, and earlier tests showed that chunk size, chunk overlap, and the number of retrieved chunks change both retrieval quality and response time [2]. For BenepiKnow, a chunk that is too small can separate a rule from the condition that explains the rule, while a chunk that is too large can add unrelated text to the prompt. The parameters therefore need to be tested on the actual dataset and questions instead of being copied from another system.

The experiment had two goals. The first goal was to find suitable chunk size, overlap, and top-k values for the BenepiKnow dataset, where top-k is the number of chunks retrieved for each question. The second goal was to measure how much the Google Colab T4 graphics processing unit (GPU) runtime reduces indexing, retrieval, and generation time compared with the central processing unit (CPU) runtime. Both runs used the same notebook, documents, questions, models, and settings, so the hardware was the only difference between the CPU and GPU results.

II. Methodology

A. Dataset and Test Questions

The knowledge base used seven Portable Document Format (PDF) files from Philippine government agencies and the World Health Organization (WHO), as listed in Table I. The notebook read each page with PyPDFLoader, removed empty pages, and removed pages with identical text, because The Philippine Family Planning Handbook [3] stores every page spread twice. Two files were scanned images without a text layer. The 13-page Republic Act (RA) No. 10354 was converted into a searchable copy with Tesseract optical character recognition (OCR) before the experiment, while the 53-page scanned copy of the Implementing Rules and Regulations (IRR) was excluded because the readable 54-page IRR contains the same rules. PhilHealth Circular No. 2024-0037 [4] also came from an OCR copy, so some extracted words of the circular contain recognition errors. After filtering, the knowledge base contained 528 pages.

TABLE I. Knowledge Base Documents

Document	Year	Pages	Text source
The Philippine Family Planning Handbook, 2023 ed.	2023	173	PDF text, duplicate pages removed
PhilHealth Circular No. 2024-0037	2024	283	OCR text layer
Implementing Rules and Regulations of RA 10354	2013	54	PDF text
Republic Act No. 10354	2012	13	Tesseract OCR before the run
DOH Administrative Order No. 2017-0002	2017	4	PDF text
PhilHealth Circular No. 2018-0005	2018	1	PDF text
Scanned copy of the RA 10354 IRR	2013	0	Excluded, same content as the readable IRR
Total		**528**	

The test set had five questions, and each question had an English, a Filipino, and a Taglish version, which gave 15 prompts. Table II lists the difficulty, topic, and source page of each question. Each question also had an expected answer and

an expected phrase from the source page. A retrieved chunk was counted as relevant only when the chunk came from the correct document and contained the full expected phrase, so a chunk that only shared keywords with the question was not counted. Four questions were easy because one passage states the answer, and Q4 was average because the answer links preferential access and free supplies to the same group of people.

TABLE II. TEST QUESTIONS

ID	Difficulty	Topic	Source and page
Q1	Easy	Right to confidentiality	Handbook, p. 26
Q2	Easy	Permanent methods that need documented consent	Handbook, p. 30
Q3	Easy	Services of free-standing clinics	Circular 2018-0005, p. 1
Q4	Average	Group with preferential and free access	DOH AO 2017-0002, p. 1
Q5	Easy	Objectives of the case rate adjustment	Circular 2024-0037, p. 2

B. Models and Hardware

The embedding model was intfloat/multilingual-e5-small [5], which converts each chunk and each question into a 384-dimensional vector and supports 94 languages, including Tagalog. The model expects a short text prefix that marks the input as a question or as a passage, so the notebook added the query prefix to every question and the passage prefix to every chunk. The vectors were stored in a new Chroma collection for every run, which kept the indexes of different configurations separate.

The proposal of BenepiKnow specified a SEA-LION embedding model, and the smaller SEA-LION-ModernBERT-Embedding-300M [6] was tested first inside Colab. The first index of 3,232 chunks did not finish after more than 60 min on the standard CPU runtime with 2 virtual CPUs (vCPUs) and after more than 28 min on the High-RAM CPU runtime with 8 vCPUs. The complete experiment builds 22 indexes, so the CPU run would have needed an estimated time of more than 10 h. The hosted SEA-LION embedding service was also unsuitable because a limit of 10 requests per minute makes the indexing time the same on any hardware. The multilingual-e5-small model therefore replaced the SEA-LION embedding model on both runtimes.

The answers came from the SEA-LION model family for Southeast Asian languages [7], specifically Qwen-SEA-LION-v4.5-27B-IT [8], through the hosted SEA-LION application programming interface (API). The API generated every answer on the servers of AI Singapore, so the Colab hardware did not affect the generation step. The notebook kept the template system prompt, a temperature of 0.2, and a limit of 256 new tokens, and the thinking mode of the model was turned off so that the whole token limit was used for the answer. The SEA-LION API allows 10 requests per minute, so the notebook waited 13 s between requests and excluded the waiting time from every measurement.

The CPU run used the Colab CPU runtime with High-RAM enabled, which provided 8 vCPUs and 51 GB of random-access memory (RAM), and the GPU run used the Colab NVIDIA Tesla T4 runtime. Table III summarizes the fixed settings. Both runs used identical copies of the notebook and the same seven files, and the notebook saved the results under an experiment identifier computed from the models, settings, questions, and pages. Results with different settings would receive a different identifier, so the CPU and GPU results could only be combined when both runs used exactly the same workload.

TABLE III. FIXED EXPERIMENTAL SETTINGS

Setting	Value
Embedding model	intfloat/multilingual-e5-small
Language model	Qwen-SEA-LION-v4.5-27B-IT through the SEA-LION API
Vector store	Chroma, new in-memory collection for every run
Generation	Temperature 0.2, 256 new tokens, thinking mode off
Timed runs	One warm-up run and three timed runs per configuration
CPU runtime	Colab CPU, High-RAM, 8 vCPUs, 51 GB RAM
GPU runtime	Colab NVIDIA Tesla T4

C. Tested Configurations

The configurations were tested in three steps that change one parameter at a time, and each configuration is written as chunk size/overlap/top-k in characters and chunks. Step 1 tested chunk sizes of 400, 800, and 1,200 characters with an overlap of 100 and a top-k of 3. Step 2 kept the best chunk size and tested overlaps of 50, 100, 150, and 200 characters, and Step 3 kept the best chunk size and overlap and tested top-k values of 1, 3, and 5. Each step ranked the results by Recall@k, then by the mean reciprocal rank (MRR@k), and then by Precision@k, and a fixed rule broke any remaining tie instead of timing, so the CPU and GPU runs selected the same values. Recall@k is the share of prompts with a relevant chunk among the top k results, MRR@k is the mean of the reciprocal rank of the first relevant chunk, and Precision@k is the share of relevant chunks among the k results.

The CPU and GPU benchmark used four configurations, namely a small configuration of 400/50/3, a medium configuration of 800/100/3, a large configuration of 1,200/150/3, and the configuration selected in Steps 1 to 3. Each configuration had one warm-up run and three timed runs on each device, and every run built a new index and answered the five English questions. The notebook recorded the indexing time, the retrieval time, the generation time, and the total response time, which is the sum of the retrieval time and the generation time of one question.

III. RESULTS AND DISCUSSION

A. Parameter Tuning Results

Table IV shows that a chunk size of 800 characters retrieved the most relevant chunks in Step 1, with a Recall@3 of 0.667 and an MRR@3 of 0.467. The 400-character chunks reached a Recall@3 of only 0.400, and a likely reason is that the 7,429 small chunks often placed passages from other documents above the expected passage. The 1,200-character chunks reached 0.467, which suggests that each long chunk mixed the expected passage with other text and lowered the similarity between the question and the chunk. The 800-character size was therefore kept for the next step.

TABLE IV. PARAMETER SELECTION RESULTS

Step	Config.	Chunks	Retrieval score		
			Recall@k	*MRR@k*	*Precision@k*
1	400/100/3	7,429	0.400	0.300	0.133
1	800/100/3	3,298	0.667	0.467	0.222
1	1,200/100/3	2,213	0.467	0.322	0.156
2	800/50/3	3,138	0.667	0.500	0.222
2	800/150/3	3,480	0.800	0.633	0.311
2	800/200/3	3,723	0.733	0.556	0.244
3	800/150/1	3,480	0.467	0.467	0.467
3	800/150/3	3,480	0.800	0.633	0.311
3	800/150/5	3,480	0.800	0.633	0.187

In Step 2, an overlap of 150 characters raised the Recall@3 from 0.667 to 0.800 and the MRR@3 from 0.467 to 0.633. A larger overlap repeats more text at the chunk boundaries, so a passage near a boundary is more likely to appear complete in at least one chunk [9]. An overlap of 200 characters lowered the Recall@3 to 0.733 and increased the number of chunks from 3,480 to 3,723, so a larger overlap did not keep improving retrieval.

In Step 3, a top-k of 1 reduced the Recall@k to 0.467, while top-k values of 3 and 5 both reached 0.800. A top-k of 3 was selected because a top-k of 5 gave the same recall with a lower Precision@k of 0.187 and passed more unrelated text to the language model. Long contexts can lower answer quality when the relevant passage is placed among many other passages [10], so the smaller top-k was the safer choice. Steps 1 to 3 gave identical scores on the CPU and GPU runtimes, which confirms that both runs retrieved from the same indexes.

Fig. 1 shows that retrieval quality depended on the language of the question. For the selected 800/150/3 configuration, the Recall@3 was 1.0 for English, 0.8 for Taglish, and 0.6 for Filipino. All source documents are written in English, so English and Taglish questions share more words with the chunks than Filipino questions. Taglish questions keep English terms such as informed consent and case rate, which explains why Taglish retrieval stayed closer to English retrieval.

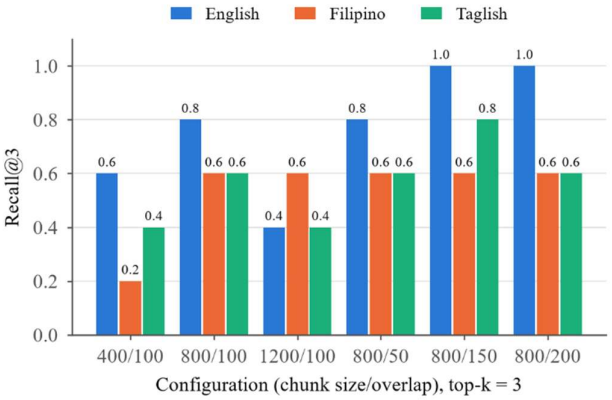


Fig. 1. Recall@3 by question language.

B. CPU and GPU Performance

Table V lists the average execution times of the four benchmark configurations. The T4 GPU indexed the documents in 25.0 s to 27.2 s, while the CPU runtime needed 231.4 s to 275.6 s, as shown in Fig. 2. Retrieval of one question took 0.015 s to 0.024 s on the GPU and 0.022 s to 0.030 s on the CPU. The three timed runs of each configuration differed by less than 2 s in indexing time on both devices, so the averages are stable.

TABLE V. AVERAGE EXECUTION TIMES ON THE CPU AND GPU RUNTIMES

Device	Config.	Average execution time (s)				Recall@3
		Indexing	*Retrieval*	*Generation*	*Response*	
CPU	400/50/3	231.37	0.030	5.31	5.34	0.6
GPU	400/50/3	26.41	0.024	5.88	5.91	0.6
CPU	800/100/3	262.03	0.025	6.25	6.28	0.8
GPU	800/100/3	25.38	0.015	5.85	5.86	0.8
CPU	1,200/150/3	264.35	0.024	6.00	6.03	0.6
GPU	1,200/150/3	25.05	0.016	6.55	6.57	0.6
CPU	800/150/3	275.60	0.022	4.59	4.61	1.0
GPU	800/150/3	27.25	0.016	6.66	6.67	1.0

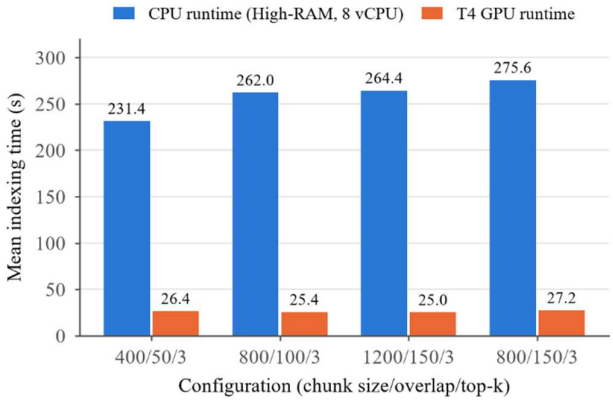


Fig. 2. Indexing time on the CPU and GPU runtimes.

The indexing time did not follow the number of chunks. The 400/50/3 configuration had the most chunks but the shortest CPU indexing time of 231.4 s, while 800/150/3 needed 275.6 s. A likely reason is that the embedding cost depends on the total amount of text, and a 150-character overlap repeats more characters in the chunks than a 50-character overlap. The speedup of each measurement was calculated with (1), where S is the speedup and T_{CPU} and T_{GPU} are the average CPU and GPU execution times.

$$S = \frac{T_{CPU}}{T_{GPU}} \quad (1)$$

A speedup greater than 1 means that the GPU was faster, and a speedup less than 1 means that the CPU was faster. Table VI shows that the indexing speedup ranged from 8.76 to 10.55 and the retrieval speedup ranged from 1.26 to 1.65. For example, the indexing speedup of 800/100/3 was 262.028 s divided by 25.378 s, which is 10.33. Indexing gained the most because multilingual-e5-small runs on the Colab device and processes thousands of chunks, while retrieval only embeds one short question and searches the index.

TABLE VI. GPU SPEEDUP BY CONFIGURATION

Configuration	GPU speedup			
	Indexing	*Retrieval*	*Generation*	*Total response*
400/50/3	8.76	1.26	0.90	0.90
800/100/3	10.33	1.65	1.07	1.07
1,200/150/3	10.55	1.50	0.92	0.92
800/150/3	10.12	1.37	0.69	0.69

The generation speedup ranged from 0.69 to 1.07, so the GPU did not make answer generation faster. The SEA-LION API generated the answers on remote servers, and the measured generation time mainly reflected network and server delays, which ranged from 3.96 s to 10.44 s between individual runs. The total response speedup followed the same values because generation took more than 99% of the response time of each question. The near-1 speedups agree with the expectation that GPU acceleration only helps the steps that run on the selected Colab hardware.

C. Answer Quality

The 15 answers of the selected 800/150/3 configuration were graded against the expected answers as correct, partially correct, or not answered. Table VII shows that 12 of the 15 answers were correct on both devices, and the CPU and GPU runs received the same grades because both runs had the same retrieval results. All five English answers were correct, and the answers in Filipino and Taglish were written in the same language as the question.

TABLE VII. ANSWER QUALITY OF THE 800/150/3 CONFIGURATION

Language	Number of answers			Cut at 256 tokens (CPU/GPU)
	Correct	*Partially correct*	*Not answered*	
English	5	0	0	0/0
Filipino	3	0	2	4/3
Taglish	4	1	0	1/1
Total	**12**	**1**	**2**	**5/4**

Two Filipino answers, Q1 and Q2, were not answered because the relevant passage was not retrieved, and the model stated that the documents did not provide enough information instead of inventing an answer. The Taglish answer to Q4 was only partially correct because the model described the poorest two quintiles of the population but did not mention the National Household Targeting System for Poverty Reduction (NHTS-PR). Faithfulness to the retrieved context is a main criterion of RAG answer evaluation [11], and none of the 30 answers added facts that were missing from the retrieved passages.

Nine of the 30 answers reached the limit of 256 new tokens and ended in the middle of a sentence, and seven of the nine were Filipino answers. Filipino text needs more tokens per word than English text, so the same limit allows a shorter Filipino answer. The cut mostly removed the closing explanation, and the main facts of Q4 and Q5 still appeared before the cut.

IV. RECOMMENDED CONFIGURATION

The recommended configuration for BenepiKnow is a chunk size of 800 characters, an overlap of 150 characters, and a top-k of 3, with indexing on the T4 GPU runtime. In terms of retrieval quality and answer quality, 800/150/3 had the highest Recall@3 of 0.800 and MRR@3 of 0.633 among all tested configurations, and 12 of the 15 answers of the configuration were correct and supported by the retrieved passages. A configuration with a faster CPU indexing time, such as 400/50/3, was not selected because the configuration retrieved fewer relevant chunks.

In terms of execution speed, the T4 GPU built the index in 27.2 s instead of 275.6 s, while the answers took 4.6 s to 6.7 s on either device because generation runs on the SEA-LION API. Both runtimes had enough memory for multilingual-e5-small, but the T4 GPU runtime consumes more Colab compute units per hour than the CPU runtime. The GPU is therefore worth the extra resources when the index must be rebuilt often, such as after adding new documents, while the CPU runtime is enough for answering questions from an index that is already built.

V. LIMITATIONS AND FUTURE WORK

The test set had only five questions, and four of the questions were easy, so the results cannot show how the configurations handle questions that need several passages. Relevance was judged by an exact expected phrase, which can miss a chunk that states the same fact with different words. The generation times depended on the network and the SEA-LION API servers, the CPU baseline used a High-RAM runtime with 8 vCPUs instead of the standard 2 vCPUs, and the limit of 256 new tokens cut nine answers. The knowledge base also depended on OCR text for two documents, and multilingual-e5-small replaced the SEA-LION embedding model of the proposal, so the retrieval results do not represent the planned SEA-LION embedding.

Future runs should use the larger test set of the proposal, with separate development and evaluation questions that include unanswerable and out-of-scope requests. A higher token limit or a word limit in the system prompt can prevent cut answers, and a relevance check based on evidence units can replace exact phrase matching. The SEA-LION embedding model can be tested again on the T4 GPU, where indexing is fast enough, to check whether the model improves Filipino retrieval. Running the CPU and GPU sessions at the same time of day can also reduce the effect of API delays on the generation measurements.

VI. CONCLUSION

The experiment selected 800/150/3 as the most suitable configuration for the BenepiKnow knowledge base and showed that the T4 GPU reduces indexing time by 8.76 to 10.55 times compared with the 8-vCPU CPU runtime. Retrieval became slightly faster on the GPU, but answer generation did not change because the language model runs on the SEA-LION API. The selected configuration answered all English questions correctly, while Filipino questions had the lowest retrieval quality.

Several problems appeared during the experiment and changed the setup. The limit of 10 requests per minute of the hosted SEA-LION embedding made the service unusable for a hardware comparison, and the local SEA-LION-ModernBERT-Embedding-300M was too slow on the CPU runtime to finish within the available time. Two documents were scanned images, so the RA 10354 Act needed OCR before the run, and the first OCR text layer joined words

together and mixed the two text columns until the text layer was rebuilt line by line. The experiment showed that the choice of hardware matters most for the steps that run inside Colab, that a fair CPU and GPU comparison needs the same workload and timing method on both devices, and that retrieval for Filipino questions is the main weakness of the current pipeline.

REFERENCES

[1] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[2] X. Wang et al., "Searching for best practices in retrieval-augmented generation," in Proc. 2024 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2024.

[3] Department of Health and World Health Organization Philippines, The Philippine Family Planning Handbook, 2023 ed. Manila, Philippines, 2023.

[4] Philippine Health Insurance Corporation, "Fifty percent (50%) adjustment of select case rates," PhilHealth Circular No. 2024-0037, Pasig City, Philippines, 2024.

[5] L. Wang et al., "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, 2024.

[6] AI Singapore, "SEA-LION-ModernBERT-Embedding-300M," Hugging Face model card, 2026. [Online]. Available: https://huggingface.co/aisingapore/SEA-LION-ModernBERT-Embedding-300M

[7] R. Ng et al., "SEA-LION: Southeast Asian languages in one network," arXiv:2504.05747, 2025.

[8] AI Singapore, "Qwen-SEA-LION-v4.5-27B-IT," Hugging Face model card, 2026. [Online]. Available: https://huggingface.co/aisingapore/Qwen-SEA-LION-v4.5-27B-IT

[9] B. Smith and A. Troynikov, "Evaluating chunking strategies for retrieval," Chroma, Tech. Rep., Jul. 2024. [Online]. Available: https://research.trychroma.com/evaluating-chunking

[10] N. F. Liu et al., "Lost in the middle: How language models use long contexts," Trans. Assoc. Comput. Linguistics, vol. 12, pp. 157-173, 2024.

[11] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in Proc. 18th Conf. European Chapter of the Association for Computational Linguistics, System Demonstrations, 2024.